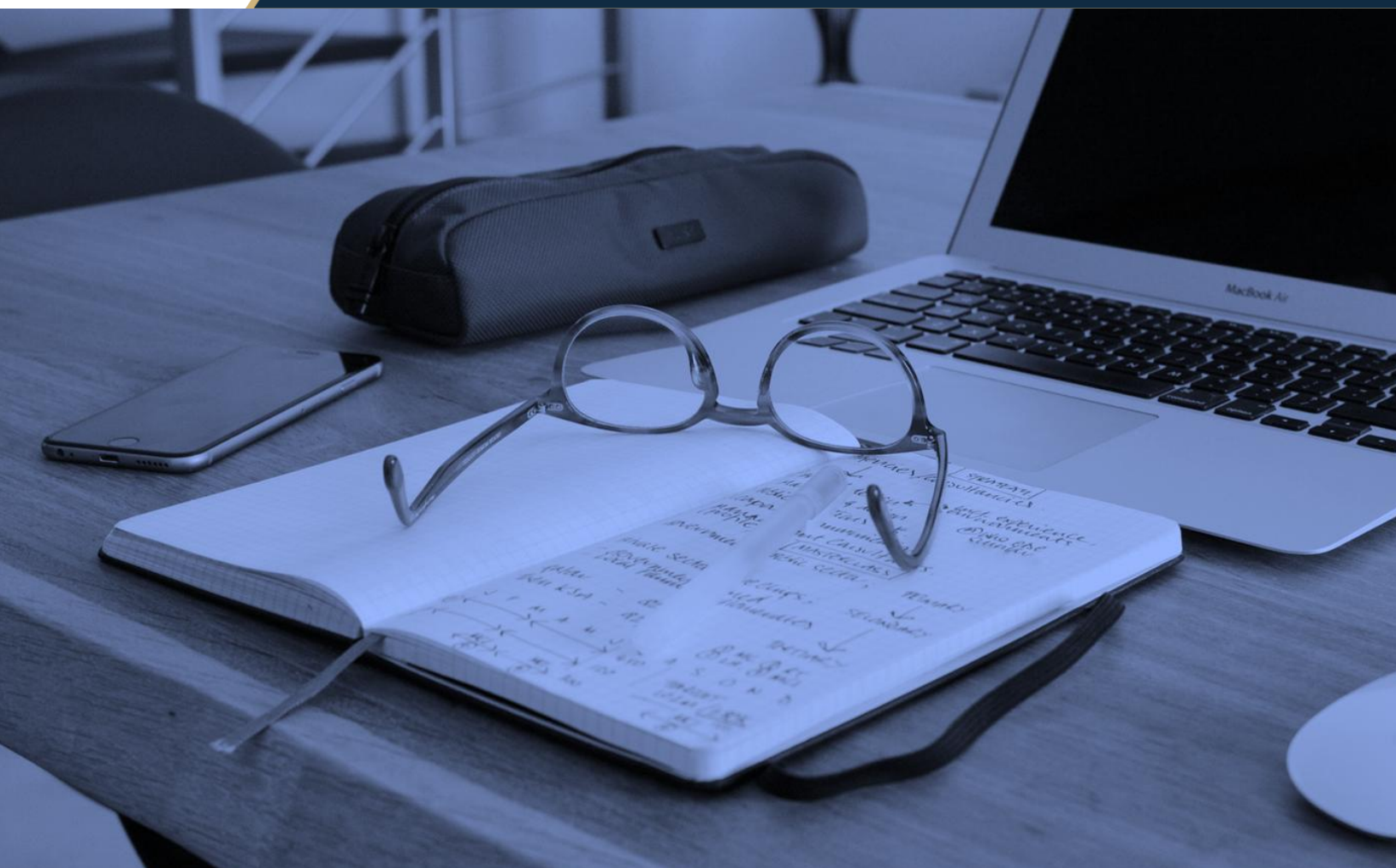




PROGRAMACIÓN PARA LA CIENCIA DE DATOS

Código: CD106ICDA

Cuaderno de Ejercitación N° 3

Depuración y optimización de algoritmos para análisis de datos

Instrucciones

Estimado/a estudiante

Te invitamos a poner a prueba tus conocimientos y habilidades en relación con los contenidos de esta Unidad 3 Depuración y optimización de algoritmos para análisis de datos. Si bien el desarrollo de este recurso formativo es opcional, te será muy útil para consolidar los aprendizajes adquiridos. Si decides ocuparlo, te recomendamos que vayas completando las actividades semana a semana, de acuerdo con lo indicado en cada apartado. Así podrás avanzar de forma paulatina, para una mejor comprensión y aplicación práctica de los distintos contenidos.

En esta ocasión, podrás ejercitar los contenidos y habilidades asociados a los siguientes indicadores de logro de la unidad.

Resultado de Aprendizaje:

- Depura los algoritmos para ajustar y mejorar la precisión y la eficiencia del análisis de datos.

Los indicadores de logro que se ejercitarán son los siguientes:

- Identifica errores y limitaciones en algoritmos de análisis de datos, aplicando técnicas de depuración para asegurar su correcto funcionamiento.
- Optimiza estructuras de datos y operaciones algorítmicas para mejorar la eficiencia computacional y reducir tiempos de ejecución.
- Ajusta parámetros y condiciones en los algoritmos, evaluando el impacto de las modificaciones en la precisión de los resultados obtenidos.
- Valida la eficacia de los algoritmos depurados mediante pruebas con diferentes conjuntos de datos, garantizando consistencia, precisión y eficiencia en el análisis.

Recuerda que puedes revisar las claves de respuesta de cada actividad al final de este recurso, comprobando cuáles fueron tus aciertos y en qué podrías mejorar. Esperamos que puedas aprovechar al máximo este recurso para el aprendizaje y que sea de utilidad para tu proceso formativo.

¡Adelante y mucho éxito!

1. Resumen de la unidad (semana 8)

Conceptos claves	Notas
Resumen	



Información complementaria

Link 1: NumPy en español (sitio oficial) – Introduce la base de la vectorización y el broadcasting en arreglos, clave para reemplazar bucles por operaciones de alto rendimiento

<https://numpy.org/es/>

Link 2: Perfiladores de Python.

<https://docs.python.org/es/3.13/library/profile.html>

Link 3: Pandas, guía en español.

<https://www.ionos.com/es-us/digitalguide/paginas-web/desarrollo-web/funcion-groupby-para-dataframes-de-pandas-en-python/>

2. Investigación y manejo de contenidos (semana 8)

2.1. Completar el cuadro comparativo.

Tema / Técnica	Objetivo	Cuándo usar	Herramientas / funciones clave	Ventajas	Riesgos / errores comunes
Depuración (debugging)					
Manejo de excepciones					
Pruebas (unitarias y regresión)					
Optimización (profiling + vectorización)					
Refactorización					

2.2 Asociación de término con sus definiciones

Instrucción: Une cada término con la definición que mejor lo describe.

Término	Definición
1. Depuración (debugging)	A. Medición detallada del tiempo/recursos por función o línea para ubicar cuellos de botella antes de optimizar.
2. Manejo de excepciones	B. Reglas que permiten operar arreglos de distinta forma/dimensión “estirando” ejes compatibles sin bucles explícitos.
3. Pruebas de regresión	C. Sustitución de bucles en Python por operaciones sobre arreglos/series implementadas en C, logrando gran velocidad y código conciso.
4. Vectorización (NumPy/pandas)	D. Proceso sistemático para identificar y corregir errores que afectan la correctitud o el flujo del programa.
5. Broadcasting (NumPy)	E. Mecanismo de control para capturar y manejar fallas previstas, evitando que el programa se interrumpa abruptamente.
6. Perfilado (profiling)	F. Conjunto de tests que verifica que los cambios no rompen comportamientos previamente correctos del sistema.

2.3 Clasificar conceptos y ejemplificar

Clasifica las siguientes 5 actividades, da un ejemplo, luego explica brevemente por qué y cómo esa actividad impacta.

Actividad	Clasificación	Ejemplo	¿Por qué impacta?	¿Cómo impacta? (mecanismo/efecto)
Depuración dirigida (breakpoints + inspección)				
Manejo de excepciones específico				
Pruebas de regresión con datos sintéticos				
Vectorización con NumPy/pandas + groupby				
Refactorización modular (DRY + funciones puras)				

3. Análisis y aplicación (semana 9)

Título del Caso / Problema: Monitoreo de flujo de pasajeros

La Dirección de Transporte de la Región Metropolitana debe monitorear a diario el flujo de pasajeros por estación para detectar cambios abruptos, planificar refuerzos operativos y reportar tendencias semanales al comité de movilidad. Para ello, dispone de archivos CSV diarios con las columnas fecha, estación y entradas, que se consolidan cada fin de mes para calcular la variación porcentual diaria por estación y construir un ranking de crecimiento. El proceso actual fue armado rápidamente por un analista y “funciona”, pero es frágil: se cae si falta un archivo, mezcla estaciones al calcular variaciones y tarda demasiado cuando supera los 5 millones de registros.

Tu misión es optimizar ese proceso sin cambiar sus objetivos. En concreto, deberás (1) corregir el cálculo de variación diaria para que compare valores dentro de cada estación, (2) implementar manejo de excepciones y registro de incidentes para que el pipeline no se detenga ante archivos ausentes o vacíos, (3) reemplazar bucles ineficientes por operaciones vectorizadas con pandas/NumPy para generar el ranking en tiempos aceptables, y (4) diseñar pruebas de regresión que certifiquen que los resultados se mantienen tras las mejoras. Al finalizar, la Dirección contará con un flujo confiable, rápido y mantenible, listo para escalar y respaldar decisiones operativas con datos de calidad.

La Dirección de Transporte de la RM te entrega archivos diarios validaciones_YYYY-MM-DD.csv con estas columnas:

- fecha (YYYY-MM-DD)
- estacion (str)
- entradas (int)

Un analista hizo un script para:

1. Cargar todos los CSV de un mes.
2. Calcular la variación porcentual diaria de entradas por estación.
3. Generar un ranking de estaciones con mayor crecimiento promedio.

Problemas actuales:

- a) Si falta un CSV de un día, el script se cae.
- b) La variación se calcula así:
 - `df['prev'] = df.sort_values('fecha')['entradas'].shift(1)`
 - `df['var_pct'] = (df['entradas'] - df['prev']) / df['prev'] * 100`

Resultado: variaciones absurdas (mezcla estaciones distintas).

- c) El ranking se calcula con un bucle fila a fila (muy lento en 5M de registros).
- d) No hay pruebas automatizadas; los cambios recientes podrían romper el resultado.

1. Detecta el error en el cálculo de la variación diaria y corrígelo.
2. Implementa manejo de excepciones para que el pipeline no se caiga si falta un CSV y registre el incidente.
3. Reemplaza el bucle que calcula el ranking por una operación vectorizada eficiente.
4. Diseña 3 pruebas de regresión para garantizar que la optimización no cambió los resultados.

Claves de Respuesta

En el siguiente QR puedes revisar las respuestas esperadas

**Referencia del presente documento:**

Instituto Profesional San Sebastian, Departamento Online (2025). Cuaderno de Ejercitación 3: *Depuración y optimización de algoritmos para análisis de datos. Programación para la ciencia de datos*. Santiago.